

Problem Solving Using C (25VBT0C103)



Module: 03

Structures, Unions, Pointers & Memory Management

Module Information

Field	Details
Course Title	Problem Solving using C
Course Code	25VBT0C103
Module Number	Module 3 of 5
Module Title	Structures, Unions, Pointers and Memory Management
Semester	I
Credits	4

Module Overview

Modules 1 and 2 equipped you to write structured C programs using variables, control flow, functions, arrays, and strings. Module 3 elevates your programming capability to the level at which real-world systems software is built. This module introduces three of the most powerful — and most nuanced — features of C: structures, unions, and pointers, along with the all-important topic of dynamic memory management.

Structures allow you to group related data of different types under a single, meaningful name. Instead of managing separate variables for a patient's ID, name, age, blood group, and admission date, a structure bundles them into a single Patient entity — exactly the kind of data modelling used in hospital management systems, banking databases, and employee payroll applications. You will learn to define structures, create nested structures, build arrays of structures, and pass structures to functions.

Unions are a memory-efficient variant of structures where all members share the same memory space. This compact design is the foundation of embedded systems programming — the microcontrollers in Tata Nexon EV's battery management system, Aadhaar biometric scanners, and industrial IoT sensors all use unions to pack multiple data interpretations into a single memory register. You will understand how unions conserve memory and where they are appropriately used.

Pointers are the heart of C programming — and the topic that most distinctly separates C from higher-level languages. A pointer is a variable that stores a memory address. Pointers enable dynamic memory allocation, efficient array manipulation, passing large data structures to functions without copying, and building complex data structures like linked lists and trees. This module covers pointer declaration, dereferencing, pointer arithmetic, pointers to arrays, and pointers to structures.

Dynamic memory management using `malloc()`, `calloc()`, `realloc()`, and `free()` allows programs to allocate memory at runtime — essential when the amount of data is not known at compile time. A patient admission system that must handle anywhere from 10 to 10,000 patients cannot pre-allocate a fixed array; it must grow its memory as needed. You will learn to use dynamic allocation correctly and understand the consequences of common errors like memory leaks and dangling pointers.

The module concludes with storage classes — the rules that determine where a variable lives in memory, how long it lives, and where it can be accessed. Understanding `auto`, `extern`, `static`, and `register` is essential for writing efficient, well-scoped programs and for understanding how larger multi-file C projects are organised.

Learning Objectives

By the end of this module, students will be able to:

- Apply the struct keyword to model real-world entities by grouping related variables of different types into a single composite data type.
- Analyse the memory layout differences between structures and unions and apply unions appropriately for memory-constrained scenarios.
- Apply pointer concepts — declaration, dereferencing, arithmetic, and pointer-to-structure — to build efficient C programs that directly manipulate memory.
- Apply dynamic memory allocation functions (malloc, calloc, realloc, free) to write programs that adapt to varying data sizes at runtime.
- Analyse the scope, lifetime, and storage location of variables using the four C storage classes: auto, extern, static, and register.

Learning Outcomes

Upon successful completion of this module, students will be able to:

- Design and implement C programs using structures and arrays of structures to model real-world entities such as bank accounts, employee records, and student information.
- Differentiate between struct and union memory layouts and implement union-based solutions for sensor data and protocol packet representation.
- Write C programs using pointers to manipulate variables, traverse arrays, and access structure members via the arrow operator (->).
- Implement programs using malloc(), calloc(), realloc(), and free() for runtime memory management, and identify and correct memory leaks and dangling pointer errors.
- Classify variables by storage class and predict their scope, lifetime, and initial value in multi-function and multi-file C programs.

Table of Topics

Sub-Unit	Topic	Coverage
3.1	Structures — Definition and Usage	struct keyword, member access (.), initialisation, typedef
3.2	Nested Structures	struct within struct, address modelling, date/time embedding
3.3	Arrays of Structures	Employee roster, student database, bank account list
3.4	Structures and Functions	Passing struct by value and by pointer, returning structs
3.5	Unions	Memory overlap, union vs struct, sensor data, protocol bytes
3.6	Pointers — Fundamentals	Declaration, address-of (&), dereference (*), NULL pointer
3.7	Pointer Arithmetic	Increment, decrement, difference, array traversal via pointer
3.8	Pointers to Arrays and Strings	Array-pointer equivalence, string manipulation via pointers
3.9	Pointers to Structures	Struct pointer, arrow operator (->), linked list intro
3.10	Dynamic Memory Allocation	malloc, calloc, realloc, free — heap vs stack
3.11	Storage Classes	auto, extern, static, register — scope, lifetime, initial value
3.12	Pointer Pitfalls and Best Practices	NULL dereference, dangling pointers, memory leaks, double free

3.1 Structures — Definition and Usage

Consider the data a bank must track for each of its customers: account number, customer name, account type, balance, branch code, and whether the account is active. Using separate variables — `long acc_no1; char name1[50]; float balance1;` — is unmanageable for even ten customers. A structure (struct) solves this by grouping all related data about one entity under a single composite type.

A structure is a user-defined data type in C that groups variables of different types under a single name. Each variable within a structure is called a member (or field).

3.1.1 Defining and Using a Structure

Scenario: Model a bank account record for HDFC Bank:

```
#include <stdio.h>
#include <string.h>

/* Structure definition: blueprint for a BankAccount */
struct BankAccount {
    long    account_no;        /* 10-digit account number */
    char    holder_name[60];   /* Account holder full name */
    char    account_type[15]; /* Savings / Current / FD */
    float   balance;           /* Current balance in INR */
    int     is_active;         /* 1 = Active, 0 = Frozen */
};

int main() {
    /* Declare a variable of type struct BankAccount */
    struct BankAccount acc1;
```

```
/* Assign values using dot (.) operator */
acc1.account_no = 5010023456L;
strcpy(acc1.holder_name, "Priya Menon");
strcpy(acc1.account_type, "Savings");
acc1.balance     = 42750.50;
acc1.is_active   = 1;

/* Display */
printf("Account No    : %ld\n", acc1.account_no);
printf("Holder       : %s\n",  acc1.holder_name);
printf("Type        : %s\n",  acc1.account_type);

printf("Balance      : Rs. %.2f\n", acc1.balance);
printf("Status       : %s\n",  acc1.is_active ? "Active"
: "Frozen");
return 0;

}
```

3.1.2 typedef — Cleaner Syntax

typedef creates an alias for a type, so you can write `BankAccount acc1` instead of `struct BankAccount acc1` — cleaner and closer to how real-world code looks in professional projects:


```
typedef struct {  
    long  account_no;  
    char  holder_name[60];  
    float balance;  
    int   is_active;  
} BankAccount; /* "BankAccount" is now the type name */  
  
int main() {  
    BankAccount acc1; /* No "struct" keyword needed */  
    acc1.account_no = 5010023456L;  
    acc1.balance     = 42750.50;  
    return 0;  
}
```

3.1.3 Structure Initialisation

```
/* Initialise at declaration - values must match member order */  
BankAccount acc2 = {5010099999L, "Rahul Sharma", "Current", 125000.0, 1};  
  
/* Designated initialisation (C99+) - order-independent, clearer */  
BankAccount acc3 = {  
    .account_no    = 5010088888L,  
    .holder_name   = "Divya Nair",  
    .balance       = 8500.0,  
    .is_active     = 1  
};
```

3.2 Nested Structures

Real-world entities often have sub-entities as members. An employee has a name, but also a date of joining — which itself has a day, month, and year. An order has an order ID, but also a delivery address — which has a flat number, street, city, and PIN code. Nested structures model this hierarchy naturally.

Scenario: A hospital's patient management system (used in Apollo Hospitals, Manipal Health) needs to store the patient's admission date and address as embedded structures:

```
#include <stdio.h>

/* Inner structure: Date */
typedef struct {
    int day;
    int month;
    int year;
} Date;

/* Inner structure: Address */
typedef struct {
    char street[80];
    char city[30];
    int  pincode;
} Address;

/* Outer structure: Patient — contains Date and Address */
typedef struct {
    int    patient_id;
    char   name[60];
    char   blood_group[5];
    Date   admission_date; /* Nested struct */
```

```
    Address home_address;    /* Nested struct */
} Patient;

int main() {

    Patient p1;

    p1.patient_id = 10042;

    strcpy(p1.name, "Arjun Reddy");
    strcpy(p1.blood_group, "B+");

    /* Access nested struct members using chained dot operator */
    p1.admission_date.day    = 15;

    p1.admission_date.month = 8;
    p1.admission_date.year  = 2024;

    p1.home_address.pincode = 500032;
    strcpy(p1.home_address.city, "Hyderabad");

    printf("Patient   : %s (ID: %d)\n", p1.name, p1.patient_id);
    printf("Admitted  : %02d/%02d/%d\n",

           p1.admission_date.day,           p1.admission_date.month,
           p1.admission_date.year);

    printf("City           : %s - %d\n",   p1.home_address.city,
           p1.home_address.pincode);

    return 0;
}
```

3.3 Arrays of Structures

A single structure models one entity. An array of structures models a collection of entities — a roster of employees, a list of bank accounts, a catalogue of products. This is the C equivalent of a database table: each row is a structure, each column is a member.

Scenario: A payroll system at Infosys needs to store and process monthly salary data for a team of 5 employees:

```
#include <stdio.h>

#include <string.h>

typedef struct {
    int emp_id;
    char name[50];
    char department[30];
    float basic_salary;
    float allowances;
    float deductions;
} Employee;

float net_salary(Employee e) {
    return e.basic_salary + e.allowances - e.deductions;
}

int main() {
    Employee team[5] = {
        {1001, "Priya Menon", "Cloud", 65000, 15000, 8200},
```

<pre>{1002,"Rahul Gupta","DevOps",72000,18000,9500},</pre>
<pre>{1003,"Sneha Iyer","QA", 58000,12000,7400},</pre>
<pre>{1004,"Arjun Das", "Data", 80000,20000,11000},</pre>
<pre>{1005,"Kavya Rao", "UI/UX", 55000,10000,6800}</pre>
<pre>};</pre> <pre>int i;</pre>
<pre>float total_payroll = 0;</pre>
<pre>printf("%-6s %-16s %-10s %10s\n","ID","Name","Dept","Net Salary");</pre>
<pre>printf("-----\n");</pre> <pre>for (i = 0; i < 5; i++) {</pre> <pre> float net = net_salary(team[i]);</pre>
<pre> total_payroll += net;</pre> <pre> printf("%-6d %-16s %-10s Rs.%8.2f\n",</pre> <pre> team[i].emp_id, team[i].name,</pre>
<pre> team[i].department, net);</pre> <pre>}</pre>
<pre>printf("-----\n");</pre> <pre>printf("Total Monthly Payroll : Rs. %.2f\n", total_payroll);</pre> <pre>return 0;</pre> <pre>}</pre>

 Let's Check

Q.No	Type	Question
1	MCQ	In C, to access a member called "balance" from a structure variable "acc", the correct syntax is: (A) acc->balance (B) acc.balance (C) acc::balance (D) balance(acc)
2	MCQ	typedef struct { int day; int month; } Date; — What does typedef do here? (A) Creates a new keyword "Date" (B) Allows "Date" to be used instead of "struct Date" (C) Makes the struct read-only (D) Allocates memory for a Date variable
3	Fill in Blank	To access a member "year" from a nested structure "dob" inside a structure variable "emp", the syntax is emp._____.year.
4	Fill in Blank	A collection of structure variables stored as a single named entity is called an _____ of structures.
5	True/False	A structure in C can contain members of different data types, including other structures. (True/False)
6	True/False	When a structure is passed to a function by value, changes made to its members inside the function affect the original structure in main(). (True/False)

Answers:

Q.No	Answer
1	(B) acc.balance — the dot (.) operator accesses members of a structure variable. The arrow (->) is used with structure pointers.
2	(B) Allows "Date" to be used instead of "struct Date" — typedef creates a type alias, simplifying declarations.
3	dob
4	Array
5	True — structures can contain members of any type: int, float, char[], double, and even other structs (nested structures).
6	False — pass-by-value creates a copy of the structure. Changes to the copy do not affect the original. To modify the original, pass a pointer to the structure.

3.4 Structures and Functions

Structures can be passed to functions by value (a copy is made) or by pointer (the original is accessible). For large structures — like a patient record with 200+ bytes of data — passing by pointer is far more efficient, since no copying occurs.

3.4.1 Passing Structure by Value

```
/* Calculates GST amount — does not need to modify the structure */
float calculate_gst(BankAccount acc, float gst_rate) {
    return acc.balance * gst_rate;    /* Works on a copy */
}
```

3.4.2 Passing Structure by Pointer

Scenario: Apply an annual interest credit to an HDFC savings account — the balance must change in the original:

```
void credit_interest(BankAccount *acc, float rate) {
    float interest = acc->balance * rate;    /* -> operator
for pointer */
    acc->balance += interest;
    printf("Interest    Rs.%.2f    credited.    New    Balance:
Rs.%.2f\n",
        interest, acc->balance);
}

int main() {
    BankAccount  myAcc  =  {5010023456L,  "Priya  Menon",
    "Savings", 42750.50, 1};
}
```



```
    credit_interest(&myAcc, 0.035); /* 3.5% p.a. SBI savings
rate */

    /* myAcc.balance is now updated */

    return 0;

}
```

💡 Did You Know?

Every row in a MySQL or PostgreSQL database table is stored internally as a C structure — called a "tuple" or "heap tuple". When you execute `SELECT * FROM customers WHERE city = 'Mumbai'`, the database engine iterates through an array of C structs in memory, comparing the city member of each struct — exactly like iterating an array of structures with a for loop. The C struct is the fundamental unit of data storage in virtually all relational databases used by Indian enterprises.

3.5 Unions

A union is similar to a structure in syntax but fundamentally different in memory layout. In a structure, every member has its own distinct memory space — the total size is the sum of all member sizes (plus padding). In a union, all members share the same memory space — the total size equals the size of the largest member. Only one member can hold a valid value at any given time.

This makes unions extremely memory-efficient for scenarios where a variable can be one of several types — but never multiple types simultaneously.

3.5.1 Union Syntax and Memory Layout

```
/* Sensor reading: could be temperature (float), humidity (int),
or
error code (char) — never more than one type at a time */

union SensorReading {
```

```
float temperature;    /* 4 bytes */
int    humidity;      /* 4 bytes */
char   error_code;    /* 1 byte   */

};

/* sizeof(union SensorReading) = 4 bytes (size of largest member)
*/

/* sizeof(struct with same members) = 9+ bytes */

int main() {

    union SensorReading sensor;

    sensor.temperature = 36.8;
    printf("Temperature: %.1f°C\n", sensor.temperature);

    /* Now store humidity – OVERWRITES the same memory */
    sensor.humidity = 72;
    printf("Humidity    : %d%%\n", sensor.humidity);

    /* sensor.temperature is now INVALID – memory was overwritten
    */

    return 0;

}
```

3.5.2 struct vs union — Side-by-Side Comparison

Feature	struct	union
Memory	Sum of all member sizes (+ padding)	Size of the largest member
Member Access	All members independently valid	Only one member valid at a time
Use Case	Group related fields that all coexist (e.g., employee record)	Type-variant data (sensor, network packet, register bits)
Real-World	Patient record, bank account, product catalogue	IoT sensor reading, CPU register overlay, TCP/IP header

3.5.3 Union for Network Packet Parsing

Scenario: A network packet in an industrial IoT gateway (like those used in smart meter reading by Tata Power) can carry either a 4-byte integer reading, a 4-byte float, or a 4-byte array of raw bytes — depending on the meter type. A union handles all three interpretations of the same 4 bytes:

<code>union MeterPacket {</code>
<code> int integer_reading; /* For digital meters */</code>
<code> float float_reading; /* For analog meters */</code>
<code> unsigned char bytes[4]; /* Raw bytes for protocol parsing */</code>
<code>};</code>

```
int main() {  
    union MeterPacket pkt;  
    pkt.float_reading = 247.35;    /* kWh reading from analog meter  
    */  
  
    printf("Float reading: %.2f kWh\n", pkt.float_reading);  
  
    /* Inspect the raw bytes in memory (little-endian on x86) */  
    printf("Raw bytes: %02X %02X %02X %02X\n",  
           pkt.bytes[0],          pkt.bytes[1],          pkt.bytes[2],  
           pkt.bytes[3]);  
    return 0;  
}
```

🔍 Analysing Misconceptions: Union Stores All Members Simultaneously

✗ MYTH: A union stores all its members independently, just like a struct — it just saves memory.

✓ REALITY: A union allocates memory equal to its LARGEST member and all members share that same memory space. Writing to one member overwrites the data of all others. At any point, only the last-written member contains a valid, meaningful value. Unions are specifically designed for mutually exclusive data — when a variable can be one type OR another, never both. Reading a member you did not last write produces undefined or garbage output.

3.6 Pointers — Fundamentals

Every variable in a C program is stored at a specific location in the computer's memory, identified by a numeric address. A pointer is a variable whose value is a memory address — it "points to" another variable. Understanding pointers is what separates a beginner C programmer from a systems programmer. They are the key to dynamic memory, efficient data processing, and building complex data structures.

Real-world analogy: Think of computer memory as a massive apartment complex. Each apartment has a unique flat number (address). A regular variable is the apartment itself — it holds data. A pointer is a piece of paper with the flat number written on it — it tells you where the data lives, not the data itself.

3.6.1 Pointer Declaration and Initialisation

```
#include <stdio.h>

int main() {

    int balance = 45000;    /* Regular variable – stores a value
    */

    int *ptr;               /* Pointer declaration – stores an
    address */

                           /* int* means "pointer to int" */

    ptr = &balance;        /* & = address-of operator */

                           /* ptr now holds the address of balance
    */

    printf("Value of balance      : %d\n",    balance);
    printf("Address of balance    : %p\n",    (void*)&balance);
    printf("Value stored in ptr   : %p\n",    (void*)ptr);
    printf("Value pointed to     : %d\n",    *ptr);
```

```
/* * = dereference operator: go to the address and read the
value */

*ptr = 50000; /* Modify balance THROUGH the pointer */
printf("balance after *ptr=50000 : %d\n", balance);

/* Output: 50000 – original variable is changed */

return 0;

}
```

3.6.2 Key Pointer Operators

Operator	Name	Meaning
&var	Address-of	Returns the memory address of variable var. Used to initialise a pointer.
*ptr	Dereference	Accesses the value stored at the address held in ptr. Also used in pointer declaration: int *ptr.
ptr = NULL	NULL pointer	Explicitly marks a pointer as pointing to "nothing". Always initialise unused pointers to NULL — prevents dangling pointer bugs.

3.6.3 Practical Example — ATM Balance Update via Pointer

Scenario: An ATM core function receives a pointer to the customer's balance and updates it after a successful withdrawal:

```
void withdraw(float *account_balance, float amount) {
    if (*account_balance >= amount) {
        *account_balance -= amount; /* Modify original balance */
        printf("Rs.%.2f dispensed. New Balance: Rs.%.2f\n",
               amount, *account_balance);
    } else {
        printf("Insufficient balance.\n");
    }
}

int main() {
    float my_balance = 12500.0;
    withdraw(&my_balance, 3000.0);
    printf("Balance in main: Rs.%.2f\n", my_balance);
    /* Output: Rs.9500.00 – updated by the function via pointer */
    return 0;
}
```

3.7 Pointer Arithmetic

Pointers support arithmetic operations, but not the same arithmetic as integers. When you increment a pointer (`ptr++`), it does not add 1 byte — it advances by the size of the data type it points to. An int pointer on a 32-bit system advances by 4 bytes per increment; a float pointer advances by 4 bytes; a double pointer by 8 bytes. This makes pointer arithmetic perfectly aligned with array element traversal.

3.7.1 Pointer Increment and Array Traversal

Scenario: A petrol pump management system (BPCL) traverses an array of daily sales figures using pointer arithmetic:

```
#include <stdio.h>

int main() {

    float daily_sales[] = {45200.0, 51300.0, 48750.0, 53100.0,
49800.0, 62400.0, 58900.0};

    float *ptr = daily_sales; /* ptr points to first element */
    /* daily_sales is equivalent to &daily_sales[0] */

    int n = 7;
    float total = 0.0;
    int day;

    printf(" Day   | Sales (Rs.)\n");
    for (day = 1; day <= n; day++) {
        printf(" %3d   | %.2f\n", day, *ptr);
        total += *ptr;
        ptr++; /* Advance pointer to next float (4 bytes forward)
    */
    }
```



```
printf("Weekly Total: Rs. %.2f\n", total);  
  
return 0;  
  
}
```

3.7.2 Pointer Arithmetic Rules

- `ptr + n`: Moves `n` elements forward. `ptr + 3` for an `int*` on a 64-bit system advances $3 \times 4 = 12$ bytes.
- `ptr - n`: Moves `n` elements backward.
- `ptr1 - ptr2`: Returns the number of elements between two pointers of the same type.
- `ptr1 == ptr2`: Compares addresses — useful for checking if two pointers point to the same location.
- `ptr[n]`: Equivalent to `*(ptr + n)` — array index notation works on any pointer.

```
int marks[] = {78, 85, 91, 70, 88};  
  
int *p = marks;  
  
printf("%d\n", *(p + 2)); /* 91 - third element */  
printf("%d\n", p[3]);    /* 70 - equivalent to *(p+3) */  
  
int *end = marks + 5;  
printf("Gap: %ld elements\n", end - p); /* 5 */
```

 Let's Check

Q.No	Type	Question
1	MCQ	int x = 10; int *p = &x; *p = 25; What is the value of x? (A) 10 (B) 25 (C) Address of x (D) Garbage
2	MCQ	A float pointer ptr points to float arr[5]. After ptr++, ptr points to: (A) The next byte in memory (B) arr[1] (4 bytes forward) (C) arr[2] (D) arr[0] still
3	Fill in Blank	The _____ operator is used to get the memory address of a variable in C.
4	Fill in Blank	An unassigned pointer should be initialised to _____ to prevent undefined behaviour.
5	True/False	ptr++ on a double pointer advances the address by 1 byte. (True/False)
6	True/False	p[2] and *(p+2) are equivalent expressions in C when p is a pointer to an array. (True/False)

Answers:

Q.No	Answer
1	(B) 25 — *p = 25 dereferences p (which holds the address of x) and stores 25 at that address. x is directly modified.
2	(B) arr[1] (4 bytes forward) — float is 4 bytes; ptr++ advances by sizeof(float) = 4 bytes, moving to the next array element.
3	& (address-of)

4	NULL
5	False — ptr++ advances by sizeof(double) = 8 bytes, not 1. Pointer arithmetic always scales by the size of the pointed-to type.
6	True — p[n] is defined as *(p + n) in C. Both expressions access the nth element relative to the pointer.

3.8 Pointers to Arrays and Strings

In C, the name of an array is essentially a constant pointer to its first element. This equivalence between arrays and pointers is one of the most important concepts in C and explains why arrays are always passed by reference to functions.

3.8.1 Array-Pointer Equivalence

```
int scores[5] = {88, 74, 92, 65, 81};

printf("%p\n", scores);           /* Address of first element */
printf("%p\n", &scores[0]);       /* Same address */
printf("%d\n", scores[2]);        /* Standard array access: 92 */
printf("%d\n", *(scores + 2));    /* Pointer arithmetic: also 92 */
```

3.8.2 String Manipulation via Pointers

Scenario: A biometric attendance system at TCS needs to extract an employee's first name from their full name string and count its characters:

```
#include <stdio.h>

int my_strlen(char *s) {
    int count = 0;
    while (*s != '\0') {    /* Walk forward until null terminator */
        count++;
        s++;
    }
    return count;
}

void print_first_name(char *full_name) {
    char *p = full_name;
    while (*p != ' ' && *p != '\0') {    /* Stop at space or end */
        putchar(*p);
        p++;
    }
    printf("\n");
}

int main() {
    char emp_name[] = "Priya Menon";
    printf("Full Name   : %s\n", emp_name);
    printf("First Name : ");
    print_first_name(emp_name);
    printf("Length      : %d chars\n", my_strlen(emp_name));

    return 0;
}
```

💡 Did You Know?

The Linux kernel's memory allocator (used in every Android phone in India) uses pointer arithmetic extensively to navigate memory blocks. When your Redmi or Samsung phone's app asks for memory, the kernel runs C code that moves a pointer forward through a free-memory list — exactly like `ptr++` in pointer arithmetic — to find the next available block. The entire 90 million lines of Linux kernel source code is C, and virtually every line that touches memory uses pointers.

3.9 Pointers to Structures

Just as pointers can point to `int`, `float`, or `char` variables, they can point to structures. A pointer to a structure stores the address of a structure variable. This is especially useful for passing large structures to functions efficiently, and is the foundation for building linked lists — one of the most important data structures in computer science.

3.9.1 The Arrow Operator (->)

When accessing structure members through a pointer, use the arrow operator (`->`) instead of the dot operator (`.`). `ptr->member` is equivalent to `(*ptr).member` — the arrow dereferences and accesses in one step.

```
#include <stdio.h>

#include <string.h>

typedef struct {
    int    emp_id;
    char   name[50];
    float  salary;
} Employee;

void give_appraisal(Employee *emp, float increment) {
```

```
emp->salary += increment;

/* Equivalent to: (*emp).salary += increment; */
printf("Appraisal applied. %s's new salary: Rs.%.2f\n",
      emp->name, emp->salary);
}

int main() {
    Employee e1 = {2201, "Kiran Patel", 68000.0};
    Employee *eptr = &e1;

    /* Access via pointer */
    printf("Before appraisal: %s earns Rs.%.2f\n",
          eptr->name, eptr->salary);
    give_appraisal(&e1, 8000.0);
    printf("After in main    : Rs.%.2f\n", e1.salary);
    /* e1.salary is now 76000 – modified through pointer */
    return 0;
}
```

3.9.2 Self-Referential Structure — Introduction to Linked List

A self-referential structure contains a pointer to its own type — this is how linked lists, trees, and other dynamic data structures are built. Each node holds data and a pointer to the next node:

```
/* Linked list node for a queue of ATM transactions */
typedef struct Transaction {
    int    txn_id;

    float  amount;

    char   type[10];          /* "DEBIT" or "CREDIT" */
    struct Transaction *next; /* Pointer to next node */
} Transaction;

int main() {
    /* Create two transaction nodes */
    Transaction t1 = {1001, 5000.0, "DEBIT", NULL};
    Transaction t2 = {1002, 2500.0, "CREDIT", NULL};

    t1.next = &t2;    /* Link: t1 -> t2 -> NULL */

    /* Traverse the list */
    Transaction *curr = &t1;
    while (curr != NULL) {
        printf("TXN %d: %s Rs.%.2f\n",
               curr->txn_id, curr->type, curr->amount);

        curr = curr->next;
    }

    return 0;
}
```

3.10 Dynamic Memory Allocation

Every variable and array declared in a C program so far has been allocated on the stack — a fixed region of memory managed automatically. Stack memory is fast, but limited in size (typically 1–8 MB) and, crucially, it requires you to know the size at compile time. What if you need to allocate memory based on user input at runtime?

Dynamic memory allocation reserves memory from the heap — a large, flexible memory region managed by the programmer. The `<stdlib.h>` library provides four functions for this: `malloc()`, `calloc()`, `realloc()`, and `free()`. These functions are what allow programs to scale — a hospital system that handles 50 patients today and 500 tomorrow without recompilation.

3.10.1 `malloc()` — Allocate Raw Memory

`malloc(size)` allocates `size` bytes and returns a void pointer to the first byte. The memory is not initialised — it contains garbage values until explicitly set.

Syntax: `void* malloc(size_t size);` → **Cast to required type**

Scenario: Dynamically allocate an array of patient records for a hospital ward — the number of patients is entered at runtime:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    int id;
    char name[50];
    float temp;
} Patient;

int main() {
```



```
int n;

printf("Enter number of patients admitted today: ");

scanf("%d", &n);

/* Allocate n Patient structs on the heap */

Patient *ward = (Patient *) malloc(n * sizeof(Patient));

if (ward == NULL) {

    printf("Memory allocation failed!\n");

    return 1;

}

int i;

for (i = 0; i < n; i++) {

    ward[i].id = 1000 + i;

    printf("Enter name for patient %d: ", ward[i].id);

    scanf("%s", ward[i].name);

    ward[i].temp = 37.0 + (i * 0.3); /* Sample temperatures
*/

}

printf("\n--- Ward Report ---\n");

for (i = 0; i < n; i++)

    printf("ID:%d %-15s Temp:%.1f°C\n",

           ward[i].id, ward[i].name, ward[i].temp);

free(ward); /* Return memory to heap – ALWAYS free after use
*/

ward = NULL; /* Prevent dangling pointer */
```

```
    return 0;
}
```

3.10.2 calloc() — Allocate and Zero-Initialise

calloc(n, size) allocates memory for n elements each of size bytes, and initialises all bytes to zero. Safer than malloc() when you need predictable initial values.

Syntax: void* calloc(size_t n, size_t size);

```
/* Allocate array of 5 integers, all initialised to 0 */
/* Use case: daily transaction count — starts at 0 each morning */
int *txn_count = (int *) calloc(5, sizeof(int));
/* txn_count[0] through txn_count[4] are all 0 */
```

3.10.3 realloc() — Resize Allocated Memory

realloc(ptr, new_size) changes the size of a previously allocated block. It may move the block to a new location if needed — the returned pointer must be used (the old one may be invalid).

Scenario: A food delivery app (Swiggy) starts by allocating space for 10 active orders, then needs to expand as more orders come in:

```
int capacity = 10;
char (*orders)[60] = malloc(capacity * sizeof(*orders));

/* ... 10 orders processed ... */

/* Need 20 more slots: */
capacity = 20;
orders = realloc(orders, capacity * sizeof(*orders));
if (orders == NULL) {
```

```
printf("Realloc failed – server at capacity!\n");  
  
exit(1);  
  
}
```

3.10.4 free() — Release Memory

`free(ptr)` returns the allocated memory back to the heap. Every `malloc()/calloc()/realloc()` must have exactly one corresponding `free()`. Forgetting `free()` causes a memory leak — the program gradually consumes all available RAM.

Function	Initialises?	Input	Typical Use
<code>malloc(n)</code>	No (garbage)	Total bytes	General-purpose allocation when you will write values immediately
<code>calloc(n, sz)</code>	Yes (all zeros)	Count + size	Counters, arrays where default-zero is meaningful
<code>realloc(p, n)</code>	Preserves old	Old ptr + new size	Dynamic arrays that grow/shrink at runtime
<code>free(p)</code>	N/A	Pointer	After finished using allocated block — MANDATORY

 Let's Check

Q.No	Type	Question
1	MCQ	What does malloc() return if memory allocation fails? (A) 0 (B) -1 (C) NULL (D) A garbage address
2	MCQ	Which function allocates memory AND initialises all bytes to zero? (A) malloc (B) realloc (C) calloc (D) alloc
3	Fill in Blank	The _____ operator is used to access a structure member through a structure pointer.
4	Fill in Blank	Forgetting to call free() after malloc() causes a _____ in the program.
5	True/False	calloc(5, sizeof(int)) allocates space for 5 integers with all elements initialised to 0. (True/False)
6	True/False	After free(ptr), the pointer ptr automatically becomes NULL in C. (True/False)

Answers:

Q.No	Answer
1	(C) NULL — malloc returns NULL when it cannot satisfy the request. Always check for NULL before using the returned pointer.
2	(C) calloc — allocates $n \times \text{size}$ bytes and sets all bytes to 0. malloc allocates without initialising.
3	-> (arrow operator)

4	Memory leak
5	True — <code>calloc(n, size)</code> allocates $n * \text{size}$ bytes and initialises every byte to 0, making all integers 0.
6	False — <code>free()</code> releases the memory but does NOT set the pointer to NULL. The pointer still holds the old (now invalid) address — a dangling pointer. You must manually set <code>ptr = NULL</code> after <code>free()</code> .

3.11 Storage Classes

Every variable in C has three properties beyond its data type: scope (where it can be accessed), lifetime (how long it exists in memory), and initial value (what it holds at the start). Storage classes — `auto`, `extern`, `static`, and `register` — control all three.

3.11.1 `auto` — Default Local Variable

`auto` is the default storage class for all local variables declared inside a function. These variables are created on the stack when the function is called and destroyed when it returns. The `auto` keyword is almost never written explicitly — it is implied.

```
void process_transaction() {  
    auto int txn_counter = 0; /* "auto" is implicit - same  
as int txn_counter = 0 */  
    txn_counter++;  
    /* txn_counter is destroyed when function returns */  
}
```

3.11.2 static — Persistent Local Variable

A static local variable retains its value between function calls — it is initialised only once and persists for the lifetime of the program. Unlike auto, it is not destroyed when the function returns.

Scenario: Count the total number of login attempts at an internet banking portal (SBI Net Banking) — the counter must persist across multiple calls to the login function:

```
#include <stdio.h>

void login_attempt(int success) {
    static int attempt_count = 0; /* Initialised ONCE, persists forever */
    attempt_count++;

    if (success)
        printf("Login %d: SUCCESS\n", attempt_count);
    else
        printf("Login %d: FAILED\n", attempt_count);
}

int main() {
    login_attempt(0); /* Login 1: FAILED */
    login_attempt(0); /* Login 2: FAILED */
    login_attempt(1); /* Login 3: SUCCESS */
    return 0;
}

/* attempt_count is NOT reset between calls — persists */
```

3.11.3 extern — Share Variable Across Files

extern declares that a variable is defined in another source file. It is used in multi-file projects to share global variables. In large projects like a bank's transaction processing system with multiple .c files, extern allows common configuration constants (like INTEREST_RATE, MAX_WITHDRAWAL) to be defined once and accessed everywhere.

```
/* config.c */

float DAILY_LIMIT = 25000.0;    /* Global variable — defined
here */

/* atm_withdraw.c */

extern float DAILY_LIMIT;        /* Tells compiler: DAILY_LIMIT
defined elsewhere */

/* Now usable in this file without redefining */
```

3.11.4 register — Request CPU Register Storage

register is a hint to the compiler to store the variable in a CPU register (fastest memory — zero access time) instead of RAM. Modern compilers automatically optimise register usage and often ignore this hint. It cannot be used with the & operator (registers have no memory address).

```
void process_data(int *arr, int n) {

    register int i;    /* Compiler hint: keep loop counter in
register */

    for (i = 0; i < n; i++)

        arr[i] *= 2;

}
```

Class	Scope	Lifetime	Default Init	Stored In
auto	Local (function)	Function duration	Garbage	Stack
static	Local (function)	Program lifetime	Zero	Data segment
extern	Global (multi-file)	Program lifetime	Zero	Data segment
register	Local (function)	Function duration	Garbage	CPU register (if available)

3.12 Pointer Pitfalls and Best Practices

Pointers are powerful — but with that power comes responsibility. Incorrect pointer usage is responsible for a large proportion of software bugs and security vulnerabilities worldwide, including buffer overflows, segmentation faults, and use-after-free exploits. Understanding these pitfalls and following best practices is essential for writing safe, reliable C programs.

3.12.1 NULL Pointer Dereference

Attempting to dereference a NULL pointer causes a segmentation fault — the program crashes immediately. This is the most common C runtime error.

```
/* WRONG — DANGEROUS */  
  
int *ptr = NULL;  
  
*ptr = 100;    /* Segmentation Fault: dereferencing NULL */  
  
/* CORRECT — Always check before use */
```



```
int *ptr = malloc(sizeof(int));

if (ptr == NULL) {
    printf("Allocation failed!\n");
    return 1;
}

*ptr = 100; /* Safe */

free(ptr);

ptr = NULL;
```

3.12.2 Dangling Pointer

A dangling pointer points to memory that has already been freed. Using it produces undefined behaviour — random crashes, corrupted data, or security vulnerabilities.

```
int *ptr = malloc(sizeof(int));

*ptr = 42;

free(ptr);

/* ptr is now a DANGLING POINTER — memory was released */
/* printf("%d", *ptr); — UNDEFINED BEHAVIOUR */

/* FIX: Always set to NULL after free */

ptr = NULL;
```

3.12.3 Memory Leak

A memory leak occurs when allocated memory is never freed. The program gradually consumes more and more RAM until the system runs out. Particularly critical in long-running server applications.

```
/* MEMORY LEAK — allocation inside loop, never freed */
for (int i = 0; i < 1000; i++) {
    int *data = malloc(1024); /* 1 KB allocated each iteration */
    /* ... do something ... */
    /* FORGOT free(data) — 1000 KB leaked! */
}

/* FIX: Free at the end of each iteration */
for (int i = 0; i < 1000; i++) {
    int *data = malloc(1024);
    /* ... do something ... */
    free(data); /* Return memory before next iteration */
}
```

3.12.4 Double Free

Calling `free()` twice on the same pointer corrupts the heap allocator's internal bookkeeping and typically crashes the program with a "double free or corruption" error. Setting the pointer to `NULL` after `free()` prevents accidental double-free.

3.12.5 Best Practices Summary

- Always initialise pointers — either to a valid address or to `NULL`.
- Always check if `malloc()/calloc()` returned `NULL` before using the pointer.
- Always free every allocated block — one `free()` per `malloc()/calloc()`.
- Always set `ptr = NULL` immediately after `free(ptr)` — prevents dangling pointer access.
- Never access an array through a pointer beyond its bounds.

- Use valgrind (Linux tool) or AddressSanitizer to detect memory leaks and invalid accesses during testing.

🔍 Analysing Misconceptions: `free()` Clears the Memory

✗ MYTH: Calling `free(ptr)` sets all the freed memory bytes to zero and makes the pointer NULL automatically.

✓ REALITY: `free()` returns the memory block to the heap allocator for reuse — it does NOT zero the memory contents, and it does NOT modify the pointer variable. After `free(ptr)`, the pointer still holds the old address (now invalid), and the freed memory may contain the old data until overwritten. This is why you must always write `ptr = NULL` after `free()` and why security-sensitive code must explicitly zero sensitive data (passwords, keys) before freeing. Tools like `memset(ptr, 0, size)` can be used for this purpose.

Case Study

■ Case Study: Aadhaar Biometric Authentication System — Structures, Unions, and Dynamic Memory in India's Largest Identity Platform

Background

UIDAI (Unique Identification Authority of India) manages the Aadhaar identity database for 1.36 billion residents. Aadhaar-based authentication is used for PDS ration distribution, Direct Benefit Transfers (DBT), income tax e-filing, SIM card activation, and PMJDY bank account opening. The core authentication system — which processes over 80 million transactions per day — is built on a high-performance C/C++ backend that uses structures, unions, and dynamic memory allocation to handle biometric and demographic data efficiently.

Module 3 Concepts Applied

- **Structures for Identity Record:** Each Aadhaar record is modelled as a nested structure: a `BiometricRecord` struct contains fingerprint minutiae (int array), iris scan data (float array), and a `Demographics` struct (name, DOB, address, and a nested `Address` struct for pincode/state/district). This mirrors the XML schema defined in UIDAI's Authentication API specification (auth.uidai.gov.in). By using a single struct, the authentication engine can pass an entire identity record to a verification function in one pointer dereference — no copying of 200+ bytes.
- **Union for Biometric Mode:** Aadhaar supports three authentication modalities — fingerprint, iris scan, or OTP. A union `BiometricData { Fingerprint fp; IrisScan iris; OTPData otp; }` is used in the authentication packet. Since only one modality is used per transaction, the union saves memory by overlaying all three representations in the same buffer space. A separate enum/int flag indicates which mode is active.

- **Dynamic Memory for Variable-Length Data:** Fingerprint minutiae count varies per individual (70–150 points). The system uses `malloc(n * sizeof(MinutiaePoint))` to allocate exactly the required memory at authentication time — no waste, no pre-allocated fixed maximum. The pointer is passed through the entire processing pipeline via struct pointers and freed after the transaction completes.
- **Pointer to Structure for Processing Pipeline:** The six-stage authentication pipeline (Receive → Decrypt → Match → Validate → Log → Respond) passes a single `AuthContext *ctx` pointer through all stages. No data is copied — each stage reads and writes to the same struct via `->` access. This eliminates 200+ bytes of struct copying per stage × 80 million transactions = significant memory and CPU savings.
- **Static Variables for Session Counters:** Each authentication thread uses a static `int daily_auth_count = 0` variable that persists across calls within the thread lifetime. This counter is used for rate-limiting (UIDAI limits authentication requests per registered AUA — Authentication User Agency — to prevent brute-force attacks). The static storage class ensures the counter is not reset between authentication calls.

Outcome

- UIDAI reports 99.97% uptime for the Aadhaar authentication service (Annual Report 2023–24), processing peaks of 2 million transactions per hour during DBT disbursement windows at month-end.
- The struct-pointer pipeline design reduced average authentication latency from 580 ms (initial design with struct copying) to 210 ms (pointer-based pipeline) — a 64% reduction, directly enabling sub-500ms SLA compliance.

Reference: UIDAI Annual Report 2023–24; UIDAI Authentication API Specification v2.5, uidai.gov.in; RBI Digital Payments Index 2024; NASSCOM Government Digital Infrastructure Report 2024.

Summary

Module 3 has taken your C programming from basic program construction to the level of professional systems programming — capable of modelling real-world entities, managing memory dynamically, and building efficient, scalable software.

Structures and Unions:

- A struct groups related variables of different types under one name. Members are accessed using the dot (.) operator. typedef simplifies the syntax. Nested structures model hierarchical entities (patient with address and admission date). Arrays of structures model entity collections (employee roster, product catalogue).
- Functions can receive structures by value (local copy, safe, slow for large structs) or by pointer (efficient, modifies original). Pass large structures by pointer in professional code.
- A union shares memory among all members — only the last-written member is valid. Size equals the largest member. Used for memory-efficient type-variant data in embedded systems, network protocols, and sensor interfaces.

Pointers:

- A pointer stores a memory address. & gives the address of a variable; * dereferences a pointer to access the value at that address. Always initialise pointers to NULL if not immediately assigned.
- Pointer arithmetic scales by the size of the pointed-to type: ptr++ on an int* advances 4 bytes. Array names are constant pointers to their first element — arr[i] and *(arr + i) are equivalent.

- A struct pointer uses `->` to access members: `ptr->member` is cleaner than `(*ptr).member`. Self-referential structures (containing a pointer to their own type) are the basis for linked lists.

Dynamic Memory:

- `malloc(n)` allocates `n` bytes on the heap (uninitialized). `calloc(n, sz)` allocates and zeros the memory. `realloc(ptr, n)` resizes an existing allocation. `free(ptr)` returns memory to the heap — mandatory after every allocation.
- Always check for `NULL` return from `malloc/calloc`. Always set `ptr = NULL` after `free()`. Never access memory beyond allocated bounds. Use tools like `valgrind` to detect leaks in larger programs.

Storage Classes:

- `auto` (default): local scope, stack, garbage initial value. `static`: local scope, data segment, zero-initialised, persists for program lifetime. `extern`: global scope across files, data segment. `register`: compiler hint for CPU register — no address, limited use in modern compilers.

Glossary

Term	Definition
Structure (struct)	A user-defined data type that groups variables of different types under a single name. Members are accessed using the dot (.) operator.
typedef	A C keyword that creates an alias for an existing data type, allowing struct types to be used without the "struct" keyword prefix.
Union	A data type where all members share the same memory space. Size equals the largest member. Only one member holds a valid value at a time.
Pointer	A variable that stores the memory address of another variable. Declared with * and initialised using the & (address-of) operator.
Dereference (*)	The operation of accessing the value stored at the memory address held by a pointer. Written as *ptr.
Arrow Operator (->)	Used to access a structure member through a pointer to that structure. ptr->member is equivalent to (*ptr).member.
malloc()	Allocates a specified number of bytes on the heap and returns a void pointer. Memory is uninitialised. Returns NULL on failure.
Memory Leak	A programming error where dynamically allocated memory is never freed. The program's memory consumption grows continuously until the system runs out of RAM.
Dangling Pointer	A pointer that refers to memory that has already been freed. Using a dangling pointer causes undefined behaviour — crashes or data corruption.
static (storage class)	A variable that retains its value between function calls and is initialised only once. Stored in the data segment rather than the stack.

Self Assessment Questions

Section A: Multiple Choice Questions (MCQ)

1. Which operator is used to access a structure member through a structure pointer?
 - A) . (dot)
 - B) -> (arrow)
 - C) :: (scope)
 - D) * (dereference)
2. What is the size of a union containing: int a (4 bytes), float b (4 bytes), char c (1 byte)?
 - A) 9 bytes
 - B) 5 bytes
 - C) 4 bytes
 - D) 1 byte
3. `int x = 50; int *p = &x; *p = 80;` What is the output of `printf("%d", x)`?
 - A) 50
 - B) 80
 - C) Address of x
 - D) 0
4. A double pointer `ptr` advances by how many bytes when `ptr++` is executed on a 64-bit system?
 - A) 1
 - B) 4
 - C) 8
 - D) 2

5. Which dynamic memory function allocates memory AND sets all bytes to zero?
- A) malloc()
 - B) alloc()
 - C) realloc()
 - D) calloc()
6. What value does malloc() return when it cannot allocate the requested memory?
- A) 0
 - B) -1
 - C) NULL
 - D) A garbage address
7. Which storage class variable retains its value between function calls?
- A) auto
 - B) register
 - C) static
 - D) extern
8. What is a "dangling pointer"?
- A) A pointer that was never initialised
 - B) A pointer that points to freed memory
 - C) A pointer that points to NULL
 - D) A pointer that is incremented beyond the array
9. Which statement correctly declares a pointer to a structure of type Student?
- A) Student ptr;
 - B) Student &ptr;
 - C) Student *ptr;
 - D) *Student ptr;

10. In a union, if you write to member "float f" and then read member "int i", what do you get?

- A) The value of f converted to int
- B) The previous value of i
- C) Undefined behaviour — f and i share the same memory
- D) Zero

11. What is the correct way to free dynamically allocated memory pointed to by ptr and prevent dangling pointer?

- A) free(ptr);
- B) free(ptr); ptr = NULL;
- C) ptr = NULL; free(ptr);
- D) delete ptr;

12. Which of the following is equivalent to arr[3] when arr is an int array?

- A) *arr + 3
- B) *(arr + 3)
- C) arr + 3
- D) &arr[3]

13. What does the extern storage class declare?

- A) A variable stored in the CPU register
- B) A constant that cannot be modified
- C) A variable defined in another source file
- D) A static variable with local scope

14. What is the output of: static int count = 0; count++; count++; printf("%d", count); when called twice?

- A) 0, 0
- B) 1, 1
- C) 1, 2

D) 2, 2

15. Nested structures in C allow:

- A) Defining a function inside a structure
- B) Including one structure as a member of another structure
- C) Creating a structure with variable-length members
- D) Sharing memory between two structures

16. A Hospital program uses `malloc(n * sizeof(Patient))`. Which header file is required?

- A) `#include <stdio.h>`
- B) `#include <string.h>`
- C) `#include <stdlib.h>`
- D) `#include <memory.h>`

17. What will happen if you dereference a NULL pointer?

- A) Returns 0
- B) Returns garbage value
- C) Causes a segmentation fault
- D) Causes a compilation error

18. In `ptr->salary`, `ptr` is a:

- A) Structure variable
- B) Structure member
- C) Pointer to a structure
- D) Integer variable

19. Why is passing a large structure to a function by pointer more efficient than passing by value?

- A) Because pointer arithmetic is faster
- B) Because no copy of the structure is made — only the address is passed
- C) Because the structure is stored in the CPU register
- D) Because structures cannot be passed by value

20. Which tool is commonly used on Linux to detect memory leaks in C programs?

- A) gdb
- B) gcc
- C) valgrind
- D) make

Section B: True / False

State whether each statement is TRUE or FALSE.

1. A union in C allows all its members to hold valid values simultaneously, just like a structure.
2. After `free(ptr)` is called, the memory pointed to by `ptr` is immediately zeroed out and unavailable for future allocation.
3. The expression `arr[i]` and `*(arr + i)` produce identical results when `arr` is an array name in C.
4. A static local variable is initialised every time the function is called.
5. Passing a pointer to a structure (pass-by-pointer) to a function allows the function to modify the original structure's members directly.

Section C: Short Answer Questions

Answer each question with a short program or explanation (3–5 sentences / code).

1. Define a structure called `Product` to represent an e-commerce product with fields: `product_id` (int), `name` (char[60]), `price` (float), and `stock_qty` (int). Write a function `void apply_discount(Product *p, float discount_pct)` that reduces the price by the given percentage. Call it from `main()` with a real product example (e.g., a phone listed on Flipkart).

-
2. Write a C program using dynamic memory allocation (malloc) to accept n employee names from the user (n entered at runtime), store them in a dynamically allocated array of strings, print all names, and free the memory properly.
 3. Explain the difference between a structure and a union with a real-world IoT sensor scenario. Include a code snippet showing memory size comparison using sizeof().
 4. Write a function void reverse_array(int *arr, int n) that reverses an integer array in-place using pointer arithmetic (no index notation like arr[i] — use only *ptr and pointer increment/decrement). Test with a sample sales data array.
 5. Explain what a memory leak is with a real-world scenario (hint: a server that handles thousands of requests). Write a code snippet showing the leak and the corrected version using free().

Self Assessment: Answer Keys

MCQ

Q.No	Correct Answer & Explanation
1	B) -> (arrow) — The arrow operator (->) is used with structure pointers. The dot (.) operator is used with structure variables.
2	C) 4 bytes — A union's size is the size of its largest member. The largest here is int (4) or float (4). All three members share those 4 bytes.
3	B) 80 — *p = 80 dereferences p (which holds the address of x) and stores 80 at that address. x is directly modified to 80.
4	C) 8 bytes — sizeof(double) = 8 on a 64-bit system. ptr++ advances by sizeof(double) = 8 bytes.
5	D) calloc() — calloc(n, size) allocates n×size bytes and sets every byte to 0. malloc() allocates without initialisation.
6	C) NULL — malloc() returns NULL when the system cannot allocate the requested memory. Always check for NULL before using the pointer.
7	C) static — a static local variable is initialised only once and retains its value between function calls. auto and register do not.
8	B) A pointer that points to freed memory — after free(ptr), the pointer still holds the old (now invalid) address. Accessing it is undefined behaviour.
9	C) Student *ptr; — pointer declarations use * before the variable name. Student ptr would be a struct variable, not a pointer.
10	C) Undefined behaviour — f and i share the same 4 bytes. Writing float then reading int re-interprets the bit pattern as int, which is not the same value. This is type-punning and is technically undefined in strict C.

11	B) <code>free(ptr); ptr = NULL;</code> — <code>free()</code> releases the memory; setting <code>ptr = NULL</code> prevents the dangling pointer from being accidentally used.
12	B) <code>*(arr + 3)</code> — <code>arr[3]</code> is defined as <code>*(arr + 3)</code> in C. <code>*arr + 3</code> adds 3 to the value; <code>arr + 3</code> is just an address; <code>&arr[3]</code> is the address of <code>arr[3]</code> .
13	C) A variable defined in another source file — <code>extern</code> tells the compiler that the variable is declared elsewhere and will be resolved by the linker.
14	C) 1, 2 — <code>static int count = 0</code> is initialised only once. First call: count becomes 1. Second call: count retains 1 and becomes 2. Not reset between calls.
15	B) Including one structure as a member of another structure — this models hierarchical entities like a Patient with a nested Address and nested Date.
16	C) <code>#include <stdlib.h></code> — <code>malloc()</code> , <code>calloc()</code> , <code>realloc()</code> , and <code>free()</code> are all declared in <code><stdlib.h></code> .
17	C) Causes a segmentation fault — the OS generates a protection fault when a program attempts to access memory at address 0 (NULL), which is reserved and unreadable.
18	C) Pointer to a structure — <code>ptr->salary</code> means <code>ptr</code> is a pointer to a structure containing a member called <code>salary</code> . The <code>-></code> operator dereferences the pointer and accesses the member.
19	B) Because no copy of the structure is made — only the address (typically 8 bytes) is passed, regardless of the structure's actual size. Passing by value would copy every byte.
20	C) <code>valgrind</code> — a Linux dynamic analysis tool that detects memory leaks, invalid reads/writes, and use-after-free errors. <code>gdb</code> is a debugger; <code>gcc</code> is the compiler; <code>make</code> is a build tool.

True / False Answer Key

Q.No	Answer / Explanation
1	FALSE — In a union, all members share the same memory space. Only the LAST written member contains a valid value. Writing to one member overwrites the data of all others. A struct is the type where all members coexist independently.
2	FALSE — <code>free()</code> releases the memory block back to the allocator but does NOT zero the contents and does NOT immediately make it "unavailable". The memory may be reallocated later, but until then it may still contain the old data. For security-sensitive data (passwords, encryption keys), you must explicitly <code>memset(ptr, 0, size)</code> before calling <code>free()</code> .
3	TRUE — This is a fundamental equivalence in C: <code>arr[i]</code> is syntactic sugar for <code>*(arr + i)</code> . The compiler generates identical machine code for both. This is why array names can be used as pointers and vice versa.
4	FALSE — a static local variable is initialised ONLY ONCE — the first time the function is called. On subsequent calls, it retains the value from the previous call. This is the key difference from auto variables, which are reinitialised each time.
5	TRUE — When a pointer to a structure is passed, the function receives the memory address of the original structure. Using the <code>-></code> operator (or <code>(*ptr).member</code>), the function reads and writes directly to the original structure's members, making changes visible in the calling function.

Short Answer Key

Q.No	Model Answer
1	<pre>typedef struct { int product_id; char name[60]; float price; int stock_qty; } Product; void apply_discount(Product *p, float pct) { p->price -= p->price * (pct/100.0); } int main() { Product ph = {1001, "Redmi Note 13 Pro", 23999.0, 150}; printf("Before: Rs.%.2f\n", ph.price); apply_discount(&ph, 10.0); /* 10% off */ printf("After : Rs.%.2f\n", ph.price); /* Rs.21599.10 */ return 0; }</pre>
2	<pre>#include <stdio.h> #include <stdlib.h> #include <string.h> int main() { int n; printf("Enter number of employees: "); scanf("%d",&n); char (*names)[50] = malloc(n * 50); if (!names) { printf("Allocation failed!\n"); return 1; } int i; for (i=0;i<n;i++) { printf("Name %d: ",i+1); scanf("%s",names[i]); } printf("\nEmployee List:\n"); for (i=0;i<n;i++) printf("%d. %s\n",i+1,names[i]); free(names); names=NULL; return 0; }</pre>
3	Struct: members each have own memory. union SensorData { float temp; int humidity; char err; }; — sizeof = 4 bytes (largest member). struct SensorData { float temp; int humidity; char err; }; — sizeof = 9+ bytes. Real-world: An IoT sensor at a smart meter (Tata Power) sends either a temperature float OR a fault code char — never both together. Using union saves 5+ bytes per packet × millions of packets = significant bandwidth savings in low-power LPWAN networks.
4	<pre>void reverse_array(int *arr, int n) { int *left = arr; int *right = arr + n - 1; while (left < right) { int temp = *left; *left = *right; *right = temp; left++; right--; } } int main() { int sales[] = {4500,6200,3800,7100,5500}; reverse_array(sales, 5); int *p = sales; while (p < sales+5) { printf("%d ",*p); p++; } /* Output: 5500 7100 3800 6200 4500 */ return 0; }</pre>

5

Memory leak: A web server (like those handling crore daily requests for Flipkart) processes each HTTP request and allocates memory for parsing headers. If `free()` is never called, each request leaks ~1 KB. After 1 million requests, 1 GB of RAM is leaked — the server crashes. LEAK: `for (i=0; i<req_count; i++) { char *buf = malloc(1024); parse_request(buf); /* free(buf) forgotten */ }` FIXED: `for (i=0; i<req_count; i++) { char *buf = malloc(1024); parse_request(buf); free(buf); buf=NULL; }` The fix ensures each request's buffer is freed immediately after use.

ONLINE DEGREE PROGRAMS OFFERED

B.Com | BBA | MBA | MCA | M.Com | MA

